

## Краткое содержание и целеполагание блокнота

Этот блокнот служит практическим руководством по созданию и сравнению двух типов нейронных сетей для классификации рукописных цифр из набора данных MNIST: полносвязной нейронной сети (FCNN) и сверточной нейронной сети (CNN).

### Цели:

- Подготовка данных:** Научиться загружать, исследовать и нормализовать данные изображений (MNIST).
- Построение и обучение FCNN:** Освоить создание, компиляцию и обучение полносвязной нейронной сети с использованием Sequential API TensorFlow/Keras.
- Построение и обучение CNN:** Изучить принципы построения сверточной нейронной сети с помощью Functional API, включая сверточные слои (Conv2D) и слои пулинга (AveragePooling2D).
- Анализ производительности:** Сравнить результаты обучения FCNN и CNN, используя метрики точности, функции потерь и матрицы ошибок.
- Визуализация:** Визуализировать веса обученных моделей и карты признаков для понимания внутренних механизмов работы сетей.

К концу этого блокнота вы получите базовое понимание того, как разрабатывать, обучать и оценивать нейронные сети для задач классификации изображений.

## Импортируем библиотеки

```
import tensorflow as tf # Основная библиотека для создания и обучения нейронных сетей
from tensorflow.keras import datasets, layers, models, regularizers # Модули Keras для работы с наборами данных, слоями, моделями и регуляризаторами
import numpy as np # Библиотека для работы с массивами и математическими операциями
import random # Для генерации случайных чисел (например, для выбора случайного изображения)
import seaborn as sns # Библиотека для статистической визуализации данных
import matplotlib.pyplot as plt # Библиотека для построения графиков и визуализации
```

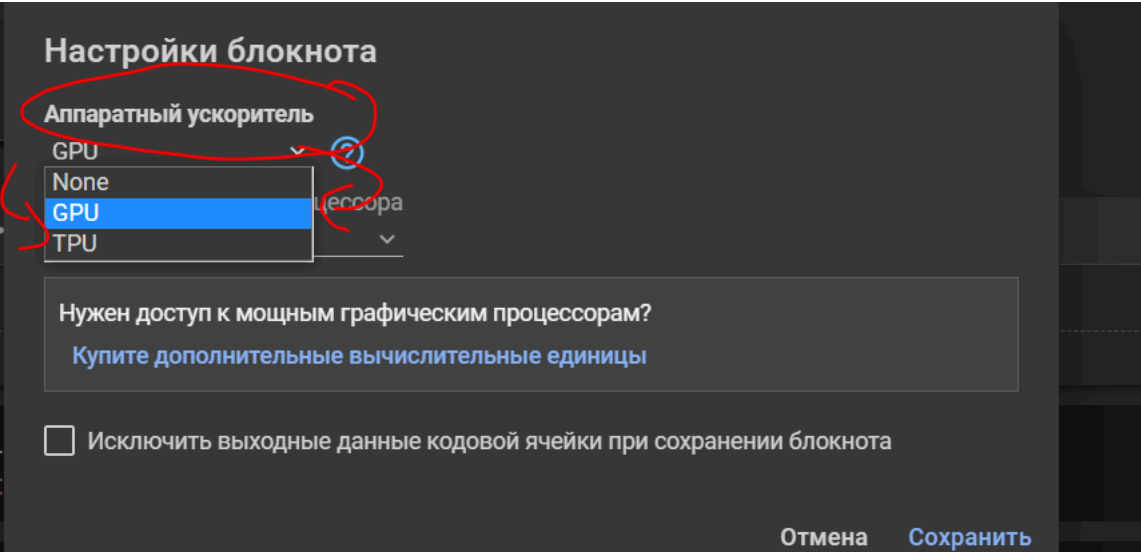
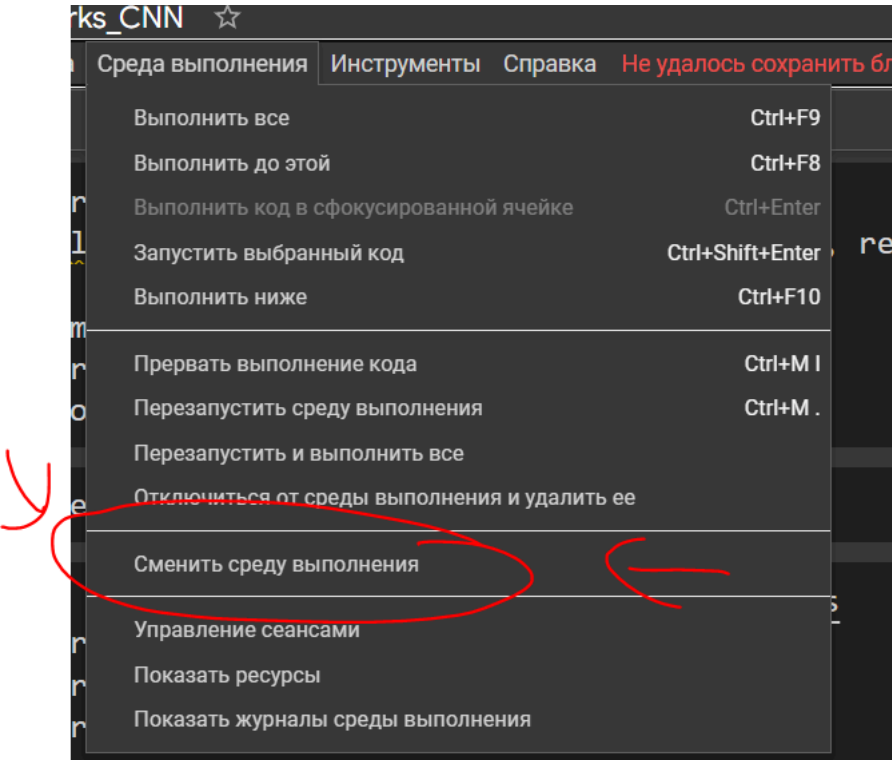
```
plt.style.use('dark_background')
```

```
!pip install -q git+https://github.com/tensorflow/docs
import tensorflow_docs as tfdocs # Библиотека для документации TensorFlow, содержит полезные утилиты
import tensorflow_docs.modeling # Модуль для моделей, вероятно, содержит утилиты для работы с моделями
import tensorflow_docs.plots # Модуль для построения графиков, вероятно, содержит утилиты для визуализации данных
```

```
Preparing metadata (setup.py) ... done
Building wheel for tensorflow-docs (setup.py) ... done
```

На всякий случай убеждаемся что подключена GPU

Этот раздел служит для проверки того, что графический процессор (GPU) подключен и доступен для использования библиотекой TensorFlow. Использование GPU значительно ускоряет обучение нейронных сетей.



Проверка того что GPU подключено и видится библиотекой tensorflow

```
device_name = tf.test.gpu_device_name() # Получаем имя устройства GPU, если оно доступно.
if device_name != '/device:GPU:0': # Проверяем, было ли найдено устройство GPU.
    raise SystemError('GPU device not found') # Если GPU не найдено, вызываем ошибку.
print('Found GPU at: {}'.format(device_name)) # Если GPU найдено, выводим сообщение с его именем.
```



```
<module 'keras.datasets' from '/usr/local/lib/python3.12/dist-packages/keras/datasets/__init__.py'>
```



- Последовательный - Sequential
- Функциональный - Functional

Начнём с простого последовательного

## Простая Полносвязная Нейронная Сеть

Представьте что у вас есть список, который вы хотите последовательно наполнить

Только вместо команды `.append` мы используем команду `.add`

на текущем этапе нам интересно два слоя

- `Flatten` - выпрямитель изображения / тензора в строку. Этот слой подготавливает входные данные для полносвязных слоев, превращая двумерное изображение (или многомерный тензор) в одномерный вектор.
- `Dense` - полносвязный слой. Каждый нейрон в этом слое соединен со всеми нейронами предыдущего слоя. Это основные строительные блоки для обучения сложных зависимостей в данных.

```
# Создаём пустую Sequential модель.
model = models.Sequential()

# Явно определяем входной слой, указывая размер изображения (28x28 пикселей с 1 каналом для оттенков серого).
model.add(tf.keras.Input(shape=(28, 28, 1)))

# Поскольку у нас полносвязная нейронная сеть и мы работаем с изображениями, первым делом мы должны "выпрямить" картинку.
model.add(layers.Flatten()) # Этот слой преобразует 2D-изображение (28x28) в одномерный вектор (784 элемента).

# Добавим два скрытых слоя по 16 нейронов. В качестве функции активации пусть будут сигмоиды.
model.add(layers.Dense(16, activation='sigmoid')) # Первый полносвязный слой с 16 нейронами и сигмоидной функцией активации.
model.add(layers.Dense(16, activation='sigmoid')) # Второй полносвязный слой также с 16 нейронами и сигмоидной функцией активации.

# Последний слой - в нём должно быть столько нейронов, сколько у нас классов (от 0 до 9, т.е. 10 классов).
# Для выходного слоя классификации, когда loss функция будет иметь from_logits=True, функцию активации можно не указывать,
# так как Softmax будет применен в функции потерь или позже для получения вероятностей.
model.add(layers.Dense(10))
```

Чтобы посмотреть на структуру полученной модели достаточно вызвать метод `.summary()`

Нам покажут

- все слои какие есть
- размерность выхода каждого слоя
- сколько параметров (весов и смещений) на каждом слое

```
model.summary() # Выводит сводку структуры модели, включая слои, выходные формы и количество параметров.
```

Model: "sequential"

| Layer (type)                     | Output Shape | Param # |
|----------------------------------|--------------|---------|
| flatten ( <code>Flatten</code> ) | (None, 784)  | 0       |
| dense ( <code>Dense</code> )     | (None, 16)   | 12,560  |
| dense_1 ( <code>Dense</code> )   | (None, 16)   | 272     |
| dense_2 ( <code>Dense</code> )   | (None, 10)   | 170     |

Total params: 13,002 (50.79 KB)  
Trainable params: 13,002 (50.79 KB)  
Non-trainable params: 0 (0.00 B)

Наша модель почти готова к обучению

Последний шаг - т.н. компиляция

Для этого нам необходимо указать 3 вещи

- Оптимизатор - `optimizer`. Грубо говоря вариант градиентного спуска для обновления весов. [На выбор есть много](#). Как правило указываем "ключевое слово" из списка в формате строковой переменной. Для простоты можно выбирать из "чего-то простого".
  - `'SGD'` - простой стохастический градиентный спуск
  - `'RMSprop'` - градиентный спуск придуманный на Coursera. Но он работает
  - `'Adam'` - Объединение предыдущих двух
- Функцию потерь - `loss`. Функция которая показывает насколько плохо нейронная сеть предсказывает что-то. Несмотря на всё [разнообразие](#) в 95% случаев (грубо, но что поделаешь) выбираем из следующих трёх
  - бинарная кросс-энтропия (если классов всего два)
  - категориальная кросс-энтропия (для многих классов)
  - средняя квадратичная ошибка (для регрессии)

Для кросс-энтропий необходимо также указать, необходимо ли свести выходы нейронной сести к вероятностям или нет

(`from_logits`) В нашем случае мы никак не нормируем выход поэтому пишем - да (`True`)

- Метрики - `metrics`. Набор [метрик](#) для промежуточной оценки модели. Для классификаций различные там доли правильных ответов, и прочие ROC-AUC. Ассигасу можно указать строковой переменной. Прочие - лучше функцией.

```
model.compile(optimizer='SGD', # Указываем оптимизатор. 'SGD' (Stochastic Gradient Descent) - это алгоритм для обновления весов нейронной сети во время обучения.
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True), # Определяем функцию потерь. SparseCategoricalCrossentropy используется для задач мног
              metrics=['accuracy'],) # Метрики для оценки производительности модели. 'accuracy' - это доля правильно классифицированных образцов.
```

После этого модель готова к обучению.

По традиции - метод `.fit()`.

Нужно указать последовательно X - данные, y - правильные метки, количество эпох.

Дополнительно можно указать валидационный набор данных - чтобы сразу оценить переобучение.

Из приятного - производные все считаются за нас. Внутре.

```
history = model.fit(train_images, train_labels, # Передаем обучающие изображения и соответствующие метки.
                    validation_data=(test_images, test_labels), # Передаем тестовые данные для оценки производительности модели на неподвижных данных.
                    epochs=10) # Указываем количество эпох обучения.
```

Epoch 1/10  
1875/1875 ————— 14s 6ms/step - accuracy: 0.1326 - loss: 2.3179 - val\_accuracy: 0.2840 - val\_loss: 2.2326  
Epoch 2/10

1875/1875 — 12s 3ms/step — accuracy: 0.3189 — loss: 2.2051 — val\_accuracy: 0.4898 — val\_loss: 2.0632  
Epoch 3/10

1875/1875 — 5s 2ms/step — accuracy: 0.5123 — loss: 2.0010 — val\_accuracy: 0.6045 — val\_loss: 1.7466  
Epoch 4/10

1875/1875 — 5s 3ms/step — accuracy: 0.6041 — loss: 1.6687 — val\_accuracy: 0.6568 — val\_loss: 1.3936  
Epoch 5/10

1875/1875 — 4s 2ms/step — accuracy: 0.6584 — loss: 1.3393 — val\_accuracy: 0.7382 — val\_loss: 1.1323  
Epoch 6/10

1875/1875 — 5s 2ms/step — accuracy: 0.7257 — loss: 1.1000 — val\_accuracy: 0.7731 — val\_loss: 0.9535  
Epoch 7/10

1875/1875 — 5s 3ms/step — accuracy: 0.7709 — loss: 0.9337 — val\_accuracy: 0.8028 — val\_loss: 0.8238  
Epoch 8/10

1875/1875 — 5s 3ms/step — accuracy: 0.8032 — loss: 0.8168 — val\_accuracy: 0.8319 — val\_loss: 0.7251  
Epoch 9/10

1875/1875 — 5s 3ms/step — accuracy: 0.8262 — loss: 0.7236 — val\_accuracy: 0.8456 — val\_loss: 0.6507  
Epoch 10/10

1875/1875 — 5s 2ms/step — accuracy: 0.8402 — loss: 0.6515 — val\_accuracy: 0.8600 — val\_loss: 0.5925

Если бы мы не делали валидацию и хотели бы оценить модель на новых данных то нам поможет метод `.evaluate()`

Будет считаться как функция потерь, так и метрики

```
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2) # Оцениваем производительность модели на тестовых данных, выводя потери и точность.

print('\nTest accuracy:', test_acc)

313/313 — 1s — 2ms/step — accuracy: 0.8600 — loss: 0.5925

Test accuracy: 0.8600000143051147
```

### Истории по Эпохам

Рисуем историю обучения по эпохам

```
def plot_history(histories,y_min,y_max):
    plt.figure(figsize=(15,6))
    # тут мы используем готовую функцию от tensorflow для построения графиков истории обучения
    plotter = tfdocs.plots.HistoryPlotter(metric = 'accuracy', smoothing_std=10,)
    plotter.plot(histories)
    plt.ylim([y_min, y_max]) # подрезаем по лимитам
```

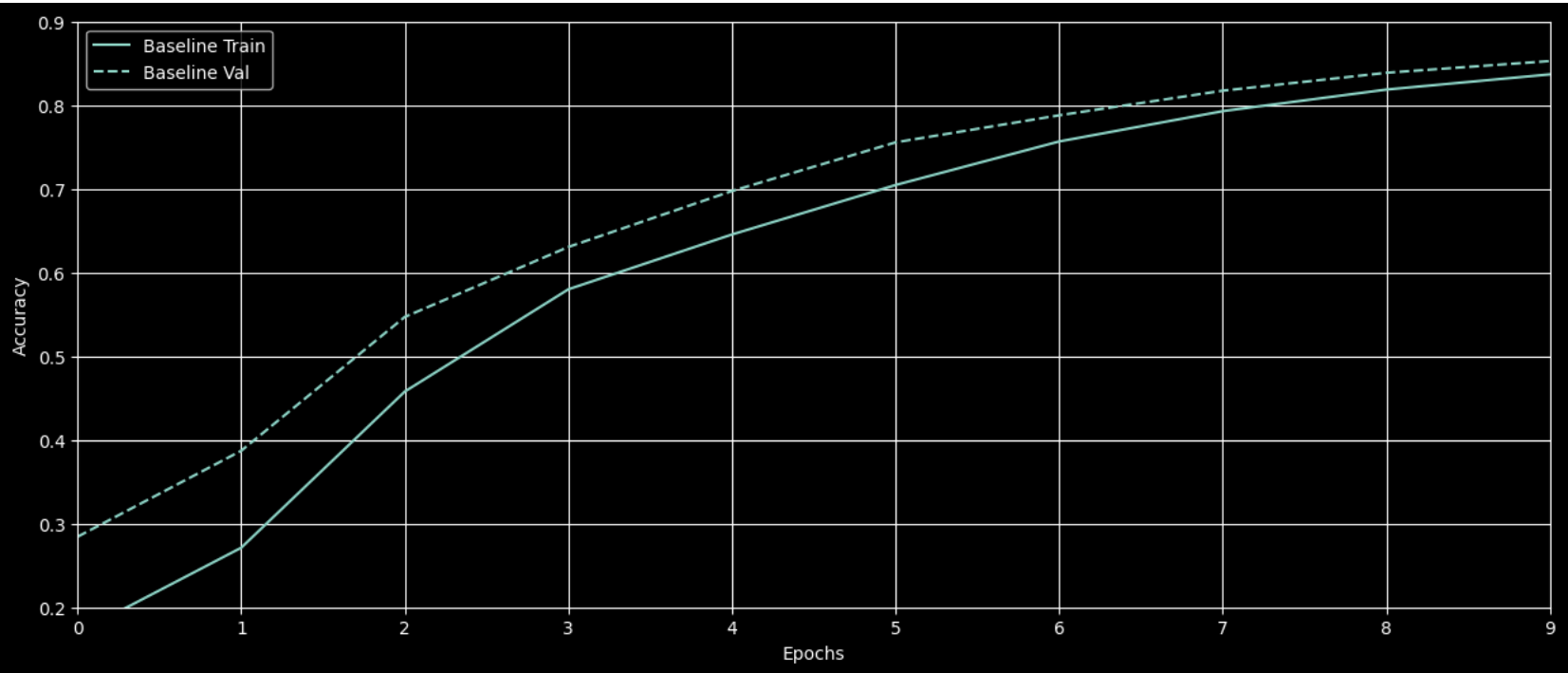
Создадим "хранилище" историй.

Чтобы потом сравнивать

```
histories = {}
```

```
histories['Baseline'] = history
```

```
plot_history(histories,0.2,0.9)
```



### Анализируем Результат

Предсказание Модели - почти традиционно через метод `.predict()`

```
prediction = model.predict(test_images)

313/313 — 1s 2ms/step
```

Но есть нюансы

На выходе не всегда понятные цифры

```
prediction[0]

array([-2.7819347 ,  1.3320314 , -2.0944417 ,  0.34920704, -0.73079836,
        0.86267924, -4.4376936 ,  4.396955 ,  0.47905836,  2.5264366 ],
      dtype=float32)
```

Преобразуем такой выход в "вероятность. С помощью функции SoftMax - нормализуем предсказания, что формально можно рассматривать как вероятность того или иного ответа (лампочка загорается или нет).

Для этого не надо переобучать модель заново. Можно создать новую модель, которая будет состоять из нашей предыдущей модели и слоя Softmax. Объединённых последовательно

```
probability_model = tf.keras.Sequential([model,
                                          tf.keras.layers.Softmax()])

# ну и делаем предсказание
```

```
prediction = probability_model.predict(test_images)
```

313/313 1s 2ms/step

Так хоть адекватнеей выглядит (хотя формально шаг можно пропустить)

```
np.round(prediction[0],3)# округлили чтобы было понятнее
```

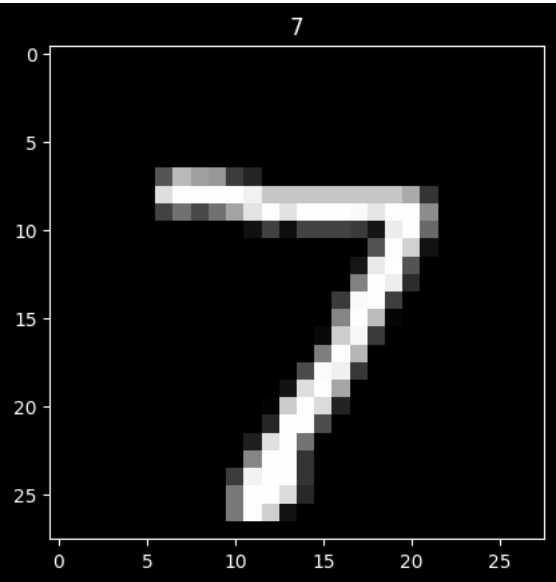
```
array([0.001, 0.037, 0.001, 0.014, 0.005, 0.023, 0.    , 0.784, 0.016,
       0.121], dtype=float32)
```

Находим индекс соответствующий максимуму предсказания

```
label_predict=np.argmax(prediction,axis=1)
label_predict[0]
```

```
np.int64(7)
```

```
# отрисуем исходное изображение
plt.imshow(test_images[0], cmap='gray', vmin=0, vmax=1)
plt.title(test_labels[0]);
```



посмотрим не только лучшее предсказание, а отсортируем возможные классы по "уверенности"

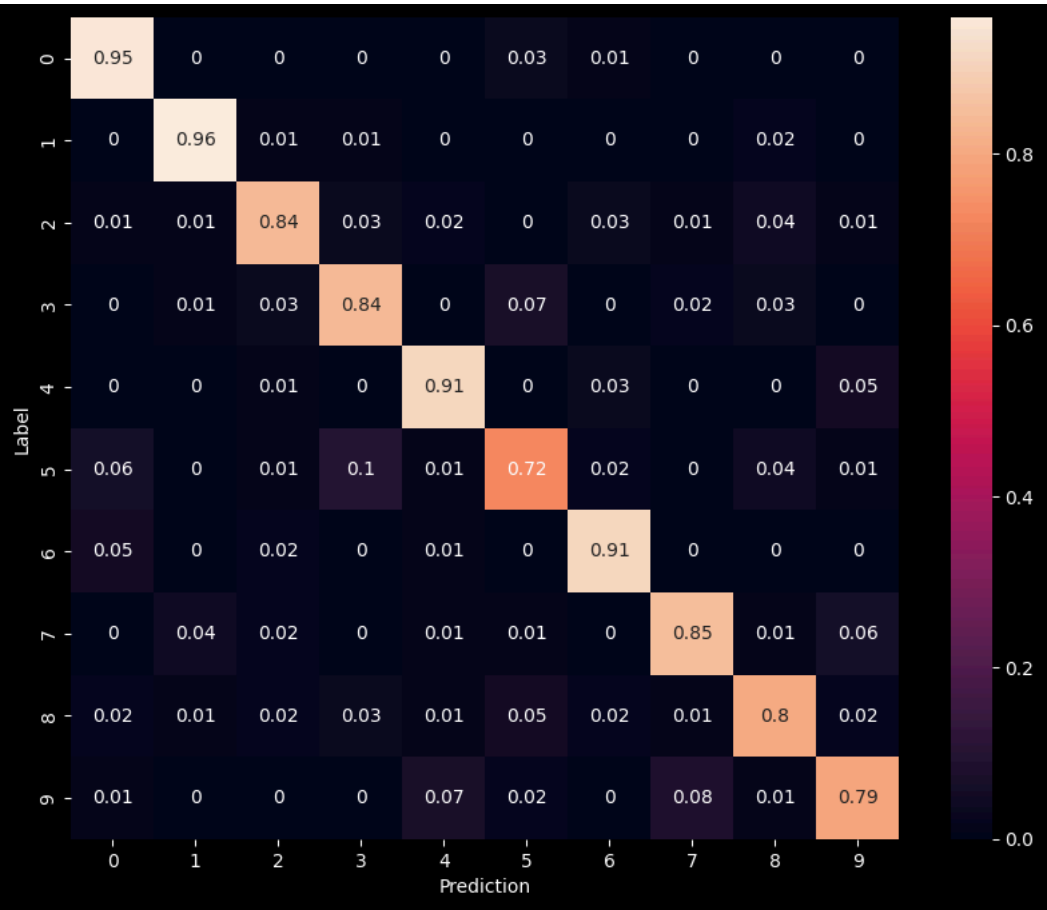
```
np.argsort(prediction[0])[::-1]
```

```
array([7, 9, 1, 5, 8, 3, 4, 2, 0, 6])
```

Матрица Ошибок

```
def plot_confusion_matrix(y_true,y_predict,labels):
    # тоже взяли из одного из примеров от tensorflow
    confusion_mtx = tf.math.confusion_matrix(y_true, y_predict,)
    confusion_mtx=np.round(confusion_mtx/np.sum(confusion_mtx,axis=1),2)
    plt.figure(figsize=(10, 8))
    sns.heatmap(confusion_mtx, xticklabels=labels, yticklabels=labels,
                annot=True, fmt='g')
    plt.xlabel('Prediction')
    plt.ylabel('Label')
    return confusion_mtx
```

```
cm=plot_confusion_matrix(test_labels,label_predict, np.unique(test_labels))
```



На ошибочные предсказания можно посмотреть через логическую индексацию

```
test_labels != label_predict
```

```
array([False, False, False, ..., False, False, False])
```



В теории можно сделать отдельный массив из ошибочных предсказаний и проанализировать их

А то вдруг ошибки из-за того что кто-то написал как курица лапой

Визуализация Весов

Визуализация обученных весов

Необходимые данные храняться в атрибуте `.trainable_variables`, обученной модели.

Отдельно веса, отдельно смещения

`model.trainable_variables` # Выводим список всех обучаемых переменных (весов и смещений) в модели. Каждая переменная представляет собой тензор, который оптимизируется в пр

```
[<Variable path=sequential/dense/kernel, shape=(784, 16), dtype=float32, value=[[ -0.0192815  -0.06936143 -0.01128425 ... -0.03645199  0.00502634
 -0.04727361]
 [ 0.08320788 -0.0839048  -0.00982711 ... -0.08167806  0.04735273
 -0.06642805]
 [ 0.03143172  0.06204621 -0.01172054 ...  0.01224495  0.02761537
  0.07260387]
 ...
 [-0.02100632  0.06621955  0.0827322  ... -0.03574095 -0.04167512
 -0.01309962]
 [ 0.02528623  0.05137531  0.04219875 ... -0.07317374 -0.00027457
 -0.043002  ]
 [ 0.08077112 -0.0493329  -0.04170591 ...  0.0413141  0.00651756
  0.02802646]]>,
<Variable path=sequential/dense/bias, shape=(16,), dtype=float32, value=[ 0.1258571  0.05328306 -0.08921862 -0.09482006 -0.21007481 -0.03932451
 0.13291594  0.04963915  0.20933472 -0.02519657 -0.02618581 -0.05203629
 -0.06305259 -0.22978486  0.04142516  0.23083279]>,
<Variable path=sequential/dense_1/kernel, shape=(16, 16), dtype=float32, value=[[ -0.65803874 -0.2977093  0.5210647  -0.29981452  0.5476765  0.5690637
 -0.6657827  -0.56445426 -0.3794575  0.4187723  0.10304744  0.0565631
 0.33453572 -0.7840576  -0.63264257  0.5282887  ]
 [ 0.5978863  1.2992957 -0.29552564 -0.2150822  -0.16983469 -0.43946123
 -0.4920028  0.12764017  0.70837  -1.6494889  -0.55049187 -0.3336361
 -0.80612797  0.8391682  0.9432964  -0.3786951  ]
 [ 0.72132313  1.229527  0.42701834 -0.19985205  0.6043704  -0.49671298
 1.0250481  -1.0310345  -0.871053  -1.4947526  -0.36035284  0.1688681
 -0.15674767  0.28218415  0.49515432 -0.6554348  ]
 [-0.32793534 -0.73120433 -0.752978  -0.69051546 -0.16460517 -0.05055775
 0.7532228  -0.1283611  0.01686867  0.16960445  0.6931168  -0.80408496
 0.91548496  1.1568209  -0.3977896  0.13210502]
 [ 0.27123985 -1.7257771  0.21423936  0.68549454  0.0278721  1.0076244
 0.6182202  -0.2756596  -0.27194488  0.6201788  0.78838485  0.8488975
 0.6895726  -0.5915409  -0.37953034 -0.3169216  ]
 [ 0.97625464  0.6109706  0.2577295  -0.22654237 -0.31634805  0.3127167
 0.20877641 -0.60279465 -0.32492474 -0.8185256  -0.31279063  0.44414848
 -0.5390712  0.4954984  0.7772381  -0.15863076]
 [-0.7565215  -0.26826212  0.68500614 -0.3490273  0.63797647  0.34976646
 -0.44013795 -0.3749344  0.279084  0.7854913  0.2519248  0.24574344
 0.19263662 -0.9399833  -0.15495087  0.43752998]
 [-0.9265205  -0.42121348 -0.61003846 -0.5681496  0.19989717 -0.33482996
 -0.05978413  0.94203687 -0.62377053  0.77305275  1.194895  -1.0588202
 0.66677564  0.65544045 -0.9420012  0.00377511]
 [-0.88652825  1.1431773  -0.8078336  -0.8700936  -0.0885141  0.93021417
 0.24607164  1.153444  -0.50827754  0.9377195  0.5261819  -0.4240313
 -0.4961289  -0.21866053 -0.6759714  0.7121606  ]
 [-0.87249  -0.4694884  1.1924843  0.13831839  0.88948816  0.3373081
 0.34496474 -1.2323195  1.2212669  -0.4502458  -0.37755358  0.67554367
 0.23302026  0.1358809  -0.4542154  -0.78835887]
 [ 0.2850396  -0.771039  0.3927234  -0.27188587  0.4692925  0.74152607
 -0.7365009  0.189735  0.8910484  -0.21137697  0.6573156  0.28710708
 0.92785037  0.41160008 -0.3338795  0.2374804  ]
 [ 0.8312715  0.48686755 -0.50837225  1.2388256  -1.0988077  -1.2325655
 0.5085043  1.0995802  -0.296369  0.7206019  -0.46024242 -0.09783302
 -1.09614  -0.5804588  0.7585325  0.38663617]
 [ 0.97417796 -0.6708361  0.7350929  1.281536  -0.04091412 -0.29358274
 -0.22375396 -0.57427764  0.4133808  0.04997022 -0.8214632  1.0651138
 -0.48956048 -0.9745251  0.6244835  -0.37410745]
 [ 0.07951096 -0.0300304  0.08214625 -0.25453237 -0.63592005 -1.0786921
 0.83994436 -0.07930456  0.37237057 -0.97899914  0.4993941  -0.93103856
 0.9047012  1.6928029  0.2945948  -0.805357  ]]
```

Возьмём просто веса (двумерные массивы)

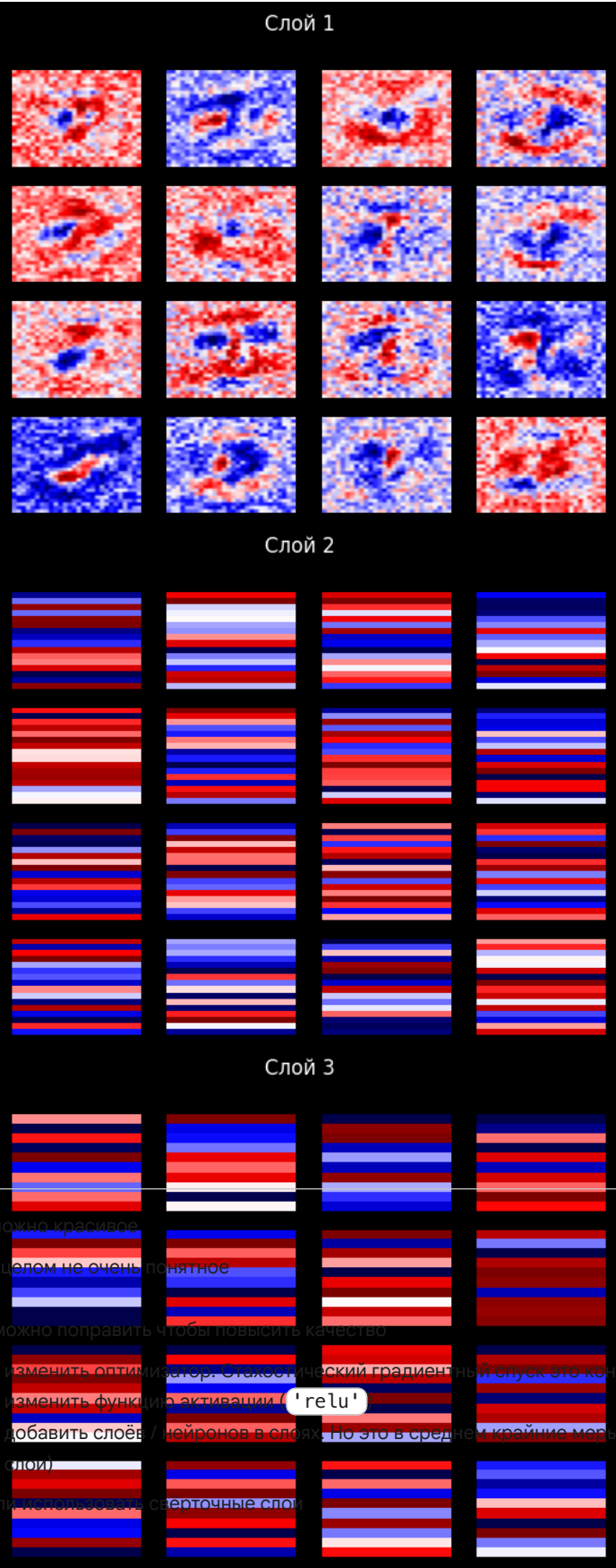
Первый слой трансформируем в псевдо-картинку (для каждого из 16 нейронов у нас есть 784 веса, которые смотрят на оригинальное изображение)

Остальные - просто разобъём на отдельные группы

```
count = 0
for weights in model.trainable_variables:
    # Проверяем, являются ли веса двумерным массивом (т.е. весами полносвязного слоя, а не смещениями).
    if len(weights.shape)>1:
        count +=1
        # Для первого слоя (входной в Flatten, затем в Dense) веса каждого нейрона имеют размерность 28x28 (размер изображения).
        # Мы преобразуем их в псевдо-изображения для визуализации.
        if count>1:
            fig, axs = plt.subplots(4, 4)
            plt.suptitle('Слой ' +str(count))
            # Для последующих слоев (после первого Dense), веса представляют связи между нейронами.
            # Их визуализируем как одномерные векторы, чтобы показать паттерны активации.
            for n, weight in enumerate(weights.numpy()):
                plt.subplot(4,4,n+1)
                plt.imshow( weight.reshape((len(weight),1)), aspect='auto', cmap='seismic' )
                plt.axis('off')

        else:
            fig, axs = plt.subplots(4, 4)
            plt.suptitle('Слой ' +str(count))
            # Веса первого Dense слоя (после Flatten) преобразуются обратно в формат 28x28 для визуализации.
            # Каждый 'псевдо-фильтр' показывает, на что 'смотрит' конкретный нейрон первого скрытого слоя.
            for n,weight in enumerate(weights.numpy().T):
                plt.subplot(4,4,n+1)
                plt.imshow( weight.reshape((28,28)), aspect='auto', cmap='seismic' )
                plt.axis('off')
```





Возможно красивое

Но в целом не очень понятное

Что можно поправить чтобы повысить качество

- изменить оптимизатор. Ставим классический градиентный спуск это конечно классика, но лучше что-то из молодежного
- изменить функцию активации ('relu')
- добавить слоёв / нейронов в слои. Но это в среднем крайние меры (сначала пытаемся увеличить нейроны, потом новые слои)

Ну или заменить сверточные слои

### Сверточная Нейронная Сеть

У нас добавляется два новых слоя

- Conv2D - двумерные свертки
- AveragePooling2D - двумерный Pooling (из прошлого). Более современный вариант - MaxPooling2D

Заодно потренируемся во втором API для формирования сеток - функциональный

Здесь мы последовательно задаём новые слои как функции от предыдущих слоёв

### Пояснение по Functional API TensorFlow

**Функциональный API** в TensorFlow/Keras предоставляет гибкий способ создания моделей, которые могут быть более сложными, чем простые последовательные (Sequential) модели. В отличие от Sequential API, где слои добавляются один за другим в линейном порядке, Functional API позволяет:

1. **Нелинейные топологии модели:** Создавать модели с несколькими входами и выходами, с ветвлениями, общими слоями и даже остаточными связями (residual connections).
2. **Общие слои:** Использовать один и тот же слой несколько раз в модели.
3. **Более явное определение потока данных:** Вы явно определяете, как выход одного слоя передается на вход другому.

Как это работает:

- Вы начинаете с определения входного слоя (tf.keras.Input), указывая ожидаемую форму данных.
- Затем вы вызываете слои как функции от предыдущих слоев. Например, output\_of\_layer1 = Layer1()(input\_layer).
- В конце вы создаете объект tf.keras.Model, передавая ему входные и выходные слои.  
tf.keras.Model(inputs=input\_layer, outputs=output\_layer).

Это позволяет создавать очень мощные и настраиваемые архитектуры нейронных сетей.

```
Input = tf.keras.Input(shape = (28,28,1)) # Определяем входной слой модели. shape=(28,28,1) указывает на размер входных изображений: 28x28 пикселей с одним каналом (оттенки серого)
conv_1 = layers.Conv2D(6, (5, 5), activation='sigmoid')(Input) # Первый сверточный слой: 6 фильтров размером 5x5. Применяется сигмоидная функция активации. В качестве входа используется Input.
pooling_1 = layers.AveragePooling2D((2, 2))(conv_1) # Первый слой пулинга (усреднения): уменьшает размерность карты признаков в 2 раза (2x2). Принимает выход conv_1.
conv_2 = layers.Conv2D(16, (5, 5), activation='sigmoid')(pooling_1) # Второй сверточный слой: 16 фильтров размером 5x5. Применяется сигмоидная функция активации. Принимает выход pooling_1.
pooling_2 = layers.AveragePooling2D((2, 2))(conv_2) # Второй слой пулинга (усреднения): уменьшает размерность карты признаков в 2 раза (2x2). Принимает выход conv_2.
flat = layers.Flatten()(pooling_2) # Слой Flatten: "выпрямляет" многомерный выход пулинга в одномерный вектор для передачи в полносвязные слои.
dense_1 = layers.Dense(16, activation='sigmoid')(flat) # Первый полносвязный слой: 16 нейронов с сигмоидной функцией активации. Принимает "выпрямленный" вектор flat.
Output = layers.Dense(10)(dense_1) # Выходной полносвязный слой: 10 нейронов, по одному на каждый класс (цифры от 0 до 9). Функция активации не указана, так как Softmax будет применена к выходу.
# В итоговой модели нужно указать кто входной слой, а кто выходной. Остальное соберется само.
conv_model = tf.keras.Model(inputs = Input, outputs = Output) # Создаем модель, указывая явно входной и выходной слои.
conv_model.summary() # Выводим сводку структуры сверточной модели.
```



Model: "functional\_5"

| Layer (type)                           | Output Shape      | Param # |
|----------------------------------------|-------------------|---------|
| input_layer_2 (InputLayer)             | (None, 28, 28, 1) | 0       |
| conv2d (Conv2D)                        | (None, 24, 24, 6) | 156     |
| average_pooling2d (AveragePooling2D)   | (None, 12, 12, 6) | 0       |
| conv2d_1 (Conv2D)                      | (None, 8, 8, 16)  | 2,416   |
| average_pooling2d_1 (AveragePooling2D) | (None, 4, 4, 16)  | 0       |
| flatten_1 (Flatten)                    | (None, 256)       | 0       |
| dense_3 (Dense)                        | (None, 16)        | 4,112   |
| dense_4 (Dense)                        | (None, 10)        | 170     |

Total params: 6,854 (26.77 KB)  
Trainable params: 6,854 (26.77 KB)  
Non-trainable params: 0 (0.00 B)

Формально весов получилось в два раза меньше

Обучаем точно так же

Для разнообразия посмотрим оптимизатор адам

Сразу стоит предупредить - свёрточные слои обучаются в среднем дольше. А без GPU вообще грустно

```
conv_model.compile(optimizer='adam', # Указываем оптимизатор. 'Adam' – это адаптивный оптимизатор, который обычно хорошо работает в большинстве случаев.  
                  loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True), # Определяем функцию потерь. SparseCategoricalCrossentropy используется для задач мног  
                  metrics=['accuracy'],) # Метрики для оценки производительности модели. 'accuracy' – это доля правильно классифицированных образцов.  
  
history_conv = conv_model.fit(train_images, train_labels, # Передаем обучающие изображения и соответствующие метки.  
                             validation_data=(test_images, test_labels), # Передаем тестовые данные для оценки производительности модели на неподвижных данных.  
                             epochs=10) # Указываем количество эпох обучения.
```

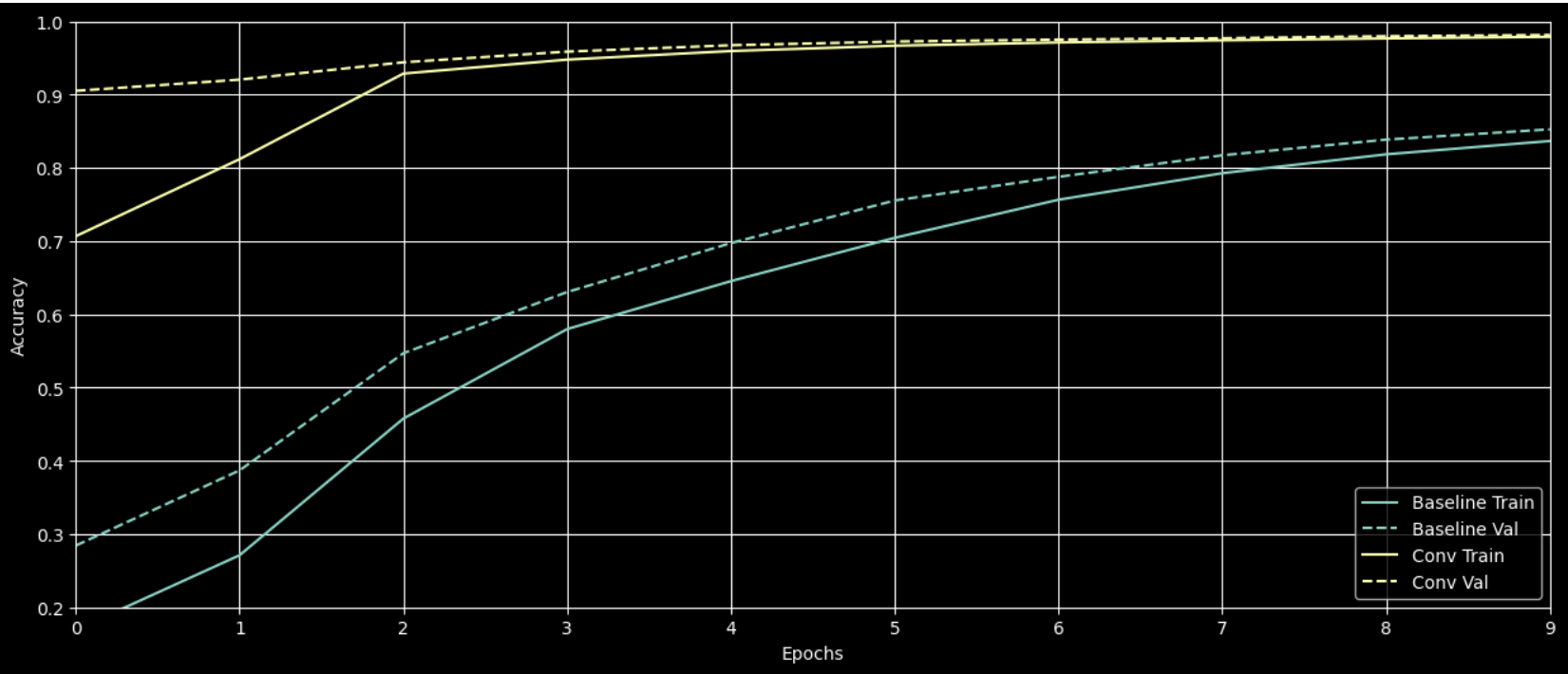
```
Epoch 1/10  
1875/1875 ————— 10s 3ms/step – accuracy: 0.4607 – loss: 1.6589 – val_accuracy: 0.9053 – val_loss: 0.4054  
Epoch 2/10  
1875/1875 ————— 6s 3ms/step – accuracy: 0.9090 – loss: 0.3642 – val_accuracy: 0.9359 – val_loss: 0.2408  
Epoch 3/10  
1875/1875 ————— 5s 3ms/step – accuracy: 0.9357 – loss: 0.2337 – val_accuracy: 0.9528 – val_loss: 0.1684  
Epoch 4/10  
1875/1875 ————— 6s 3ms/step – accuracy: 0.9522 – loss: 0.1685 – val_accuracy: 0.9650 – val_loss: 0.1270  
Epoch 5/10  
1875/1875 ————— 5s 3ms/step – accuracy: 0.9622 – loss: 0.1317 – val_accuracy: 0.9702 – val_loss: 0.1059  
Epoch 6/10  
1875/1875 ————— 6s 3ms/step – accuracy: 0.9698 – loss: 0.1063 – val_accuracy: 0.9751 – val_loss: 0.0847  
Epoch 7/10  
1875/1875 ————— 6s 3ms/step – accuracy: 0.9723 – loss: 0.0929 – val_accuracy: 0.9756 – val_loss: 0.0781  
Epoch 8/10  
1875/1875 ————— 5s 3ms/step – accuracy: 0.9751 – loss: 0.0834 – val_accuracy: 0.9787 – val_loss: 0.0709  
Epoch 9/10  
1875/1875 ————— 6s 3ms/step – accuracy: 0.9778 – loss: 0.0748 – val_accuracy: 0.9812 – val_loss: 0.0611  
Epoch 10/10  
1875/1875 ————— 5s 3ms/step – accuracy: 0.9805 – loss: 0.0680 – val_accuracy: 0.9821 – val_loss: 0.0597
```

Даже немного переобучения в конце

Сравним истории по эпохам

```
histories['Conv'] = history_conv
```

```
plot_history(histories,0.2,1.0)
```



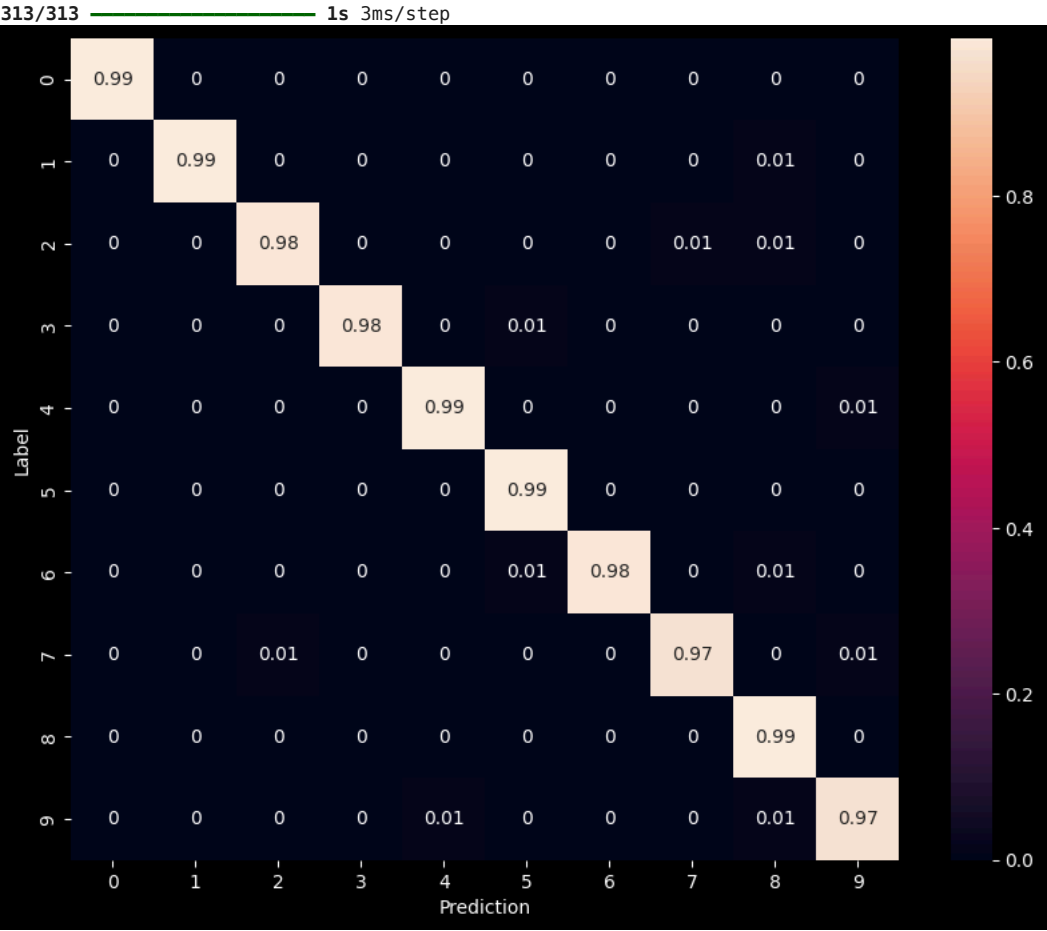
to the moon

Свёрточная сеть на 1 эпохи превосходит полносвязную в конце (на валидационных данных)

Сравним матрицу ошибок

```
prediction = conv_model.predict(test_images)  
  
label_predict=np.argmax(prediction,axis=1)  
  
cm=plot_confusion_matrix(test_labels,label_predict, np.unique(test_labels))
```





```
test_labels != label_predict  
  
array([False, False, False, ..., False, False, False])
```

Почти идеальная диагональ

### Визуализация Весов (почти)

Функция для визуализации активаций после каждого слоя

Вдохновлено блокнотом из курса DeepLearning AI: TensorFlow Developer

```
def conv_layers_vis(features_map, layer_names):  
    # Эта функция визуализирует карты признаков (активации) после каждого сверточного слоя.  
    # features_map: список карт признаков, полученных после прохода изображения через модель.  
    # layer_names: список названий слоев, соответствующих картам признаков.  
    for layer_name, feature_map in zip(layer_names, successive_feature_maps):  
        # Визуализируем только сверточные слои (те, у которых 4 измерения: батч, высота, ширина, количество фильтров).  
        if len(feature_map.shape) == 4:  
            n_features = feature_map.shape[-1] # Получаем количество фильтров (карт признаков) в текущем слое.  
            size = feature_map.shape[-2] # Получаем размер (высоту/ширину) одной карты признаков.  
  
            # Создаем пустую сетку для отображения всех карт признаков текущего слоя.  
            display_grid = np.zeros((size, size * n_features))  
  
            # предобработка весов чтобы было чуть понятнее, что там происходит  
            # Нормализуем каждую карту признаков для лучшей визуализации.  
            for i in range(n_features):  
                x = feature_map[:, :, :, i] # Выбираем карту признаков для текущего фильтра и первого изображения в батче.  
                x -= x.mean() # Вычитаем среднее значение, чтобы центрировать данные.  
                x /= x.std() # Делим на стандартное отклонение для нормализации масштаба.  
                x *= 64 # Умножаем для увеличения контрастности.  
                x += 128 # Добавляем смещение, чтобы значения были в диапазоне 0-255.  
                x = np.clip(x, 0, 255).astype('uint8') # Обрезаем значения, чтобы они были в пределах 0-255, и преобразуем в целые числа.  
                # Размещаем обработанную карту признаков в соответствующем месте на общей сетке.  
                display_grid[:, i * size : (i + 1) * size] = x  
  
            # Визуализация  
            scale = 25. / n_features # Определяем масштаб для размера графика.  
            plt.figure( figsize=(scale * n_features, scale) ) # Создаем фигуру с динамическим размером.  
            plt.axis('off') # Отключаем оси координат.  
            plt.title ( layer_name ) # Устанавливаем заголовок графика - название слоя.  
            plt.grid ( False ) # Отключаем сетку.  
            plt.imshow( display_grid, aspect='auto', cmap='seismic' ) # Отображаем сетку карт признаков, используя цветовую карту 'seismic'.
```

Создаем новую модель

Которая в качестве входа использует изображение, а в качестве выходов - выходы после каждого слоя

Используем функциональный API

```
# не берем первый слой потому что он у нас в этой модели - входной  
successive_outputs = [layer.output for layer in conv_model.layers[1:]]  
# функциональный API позволяет делать даже такие сложные конструкции  
visualization_model = models.Model(inputs = conv_model.input, outputs = successive_outputs)  
  
layer_names = [layer.name for layer in conv_model.layers[1:]]
```

Берем Случайное Изображение

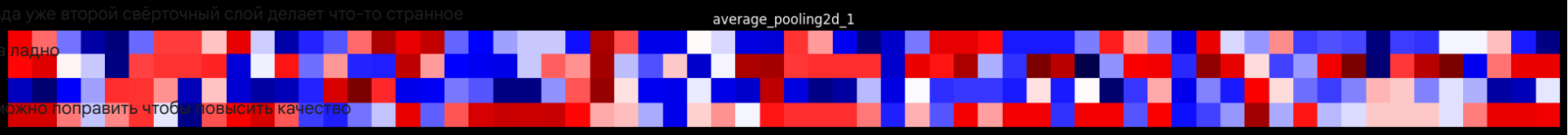
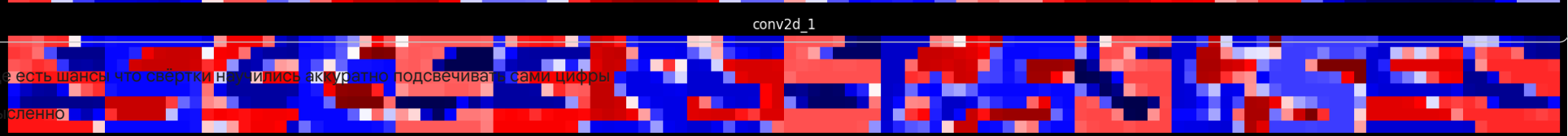
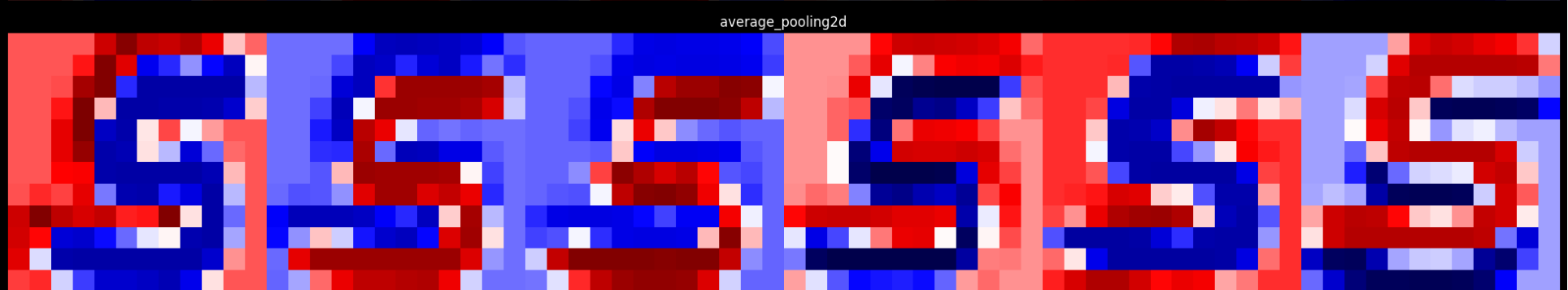
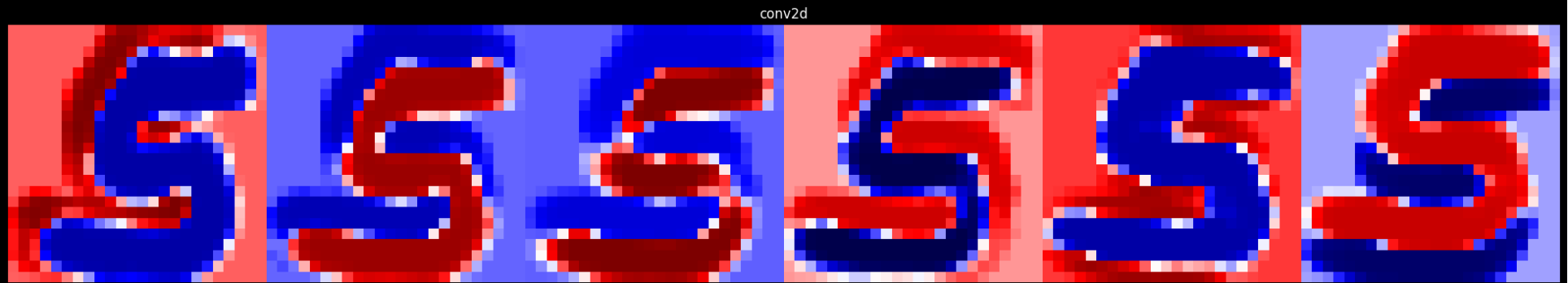
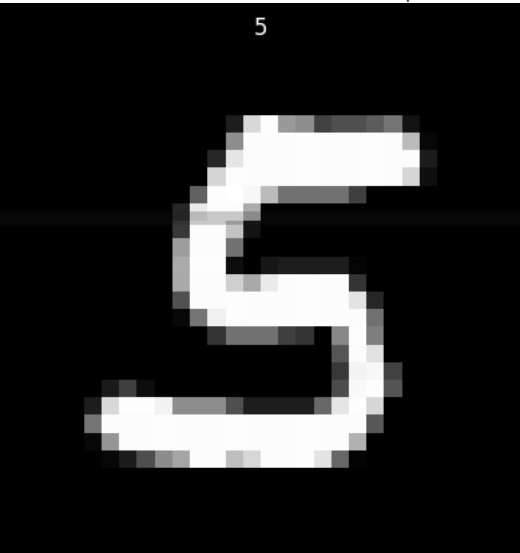
Добавляем "пустую" размерность

Смотрим что получается в итоге

```
index = random.randint(0,60000) # Выбираем случайный индекс для изображения из обучающего набора данных.  
  
image = train_images[index] # Получаем изображение по выбранному случайному индексу.  
  
plt.imshow(image, cmap='gray', vmin=0, vmax=1) # Отображаем выбранное изображение в оттенках серого.  
plt.axis('off') # Отключаем оси координат для чистоты изображения.  
plt.title(train_labels[index]) # Устанавливаем заголовок изображения, показывающий его истинную метку.  
image = image.reshape((1,) + image.shape) # Добавляем "пустую" размерность (размер батча) к изображению, чтобы оно соответствовало входному формату модели (1, 28, 28, 1).  
  
successive_feature_maps = visualization_model.predict(image) # Получаем карты признаков после каждого слоя, пропуская изображение через специальную модель для визуализации.
```

```
conv_layers_vis(successive_feature_maps, layer_names) # Вызываем функцию для визуализации полученных карт признаков.
```

1/1 — 0s 50ms/step



Вроде есть шанс, что свёртки научились аккуратно подсвечивать сами цифры

Осмеленно

Правда уже второй свёрточный слой делает что-то странное

Ну да ладно

Что можно поправить чтобы повысить качество

- изменить оптимизатор. Если с каким-то оптимизатором не получается – смените, вдруг проблема в нём
- изменить функцию активации (опять же в среднем лучше `'relu'`)
- добавить свёрточных слоёв / каналов в них. При этом нужно быть аккуратней с последним свёрточным слоем (который перед выпрямлением). Чтобы там было не слишком много параметров. А то будет не весело
- добавить полносвязных слоёв / увеличить нейроны в них. Но это так, крайние меры

### Резюме блокнота: Чему мы научились

В этом блокноте мы прошли путь от подготовки данных до создания и обучения двух типов нейронных сетей для классификации рукописных цифр из набора данных MNIST.

#### 1. Подготовка Окружения и Данных

- **Импорт библиотек:** Мы импортировали необходимые библиотеки, такие как TensorFlow/Keras для построения моделей, NumPy для работы с массивами, Matplotlib и Seaborn для визуализации, а также `tensorflow_docs` для утилит обучения.
- **Проверка GPU:** Убедились, что GPU подключено и доступно для TensorFlow, что критически важно для ускорения обучения нейронных сетей.
- **Загрузка данных MNIST:** Загрузили набор данных MNIST, состоящий из 60 000 обучающих и 10 000 тестовых изображений цифр размером 28x28 пикселей.
- **Нормализация данных:** Преобразовали значения пикселей изображений из диапазона [0, 255] в [0, 1], разделив их на 255.0. Это улучшает производительность обучения нейронных сетей.

#### 2. Построение и Обучение Полносвязной Нейронной Сети (FCNN)

- **Sequential API:** Создали модель, последовательно добавляя слои.
  - `tf.keras.Input`: Явно определили входной слой.
  - `Flatten`: Преобразовали 2D-изображения (28x28) в 1D-векторы (784 элемента).
  - `Dense` слои: Добавили два скрытых полносвязных слоя по 16 нейронов с сигмоидной функцией активации. Выходной слой имел 10 нейронов (по количеству классов).
- **Компиляция модели:** Настроили процесс обучения, указав:
  - `optimizer='SGD'`: Использовали стохастический градиентный спуск.
  - `loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True)`: Применили функцию потерь, подходящую для многоклассовой классификации с целочисленными метками.
  - `metrics=['accuracy']`: Отслеживали точность модели во время обучения.
- **Обучение модели:** Выполнили обучение с помощью метода `model.fit()` на 10 эпохах, используя валидационные данные для отслеживания переобучения.
- **Оценка и анализ:**
  - Оценили модель на тестовых данных с помощью `model.evaluate()`.



- **Предсказания:** Получили предсказания и преобразовали их в вероятности с помощью `tf.keras.layers.Softmax`, а затем извлекли предсказанные метки с помощью `np.argmax()`.
- **Матрица ошибок:** Визуализировали матрицу ошибок (Confusion Matrix) для анализа производительности модели по классам.
- **Визуализация весов:** Исследовали веса первого полносвязного слоя, чтобы понять, какие паттерны изображения активируют нейроны.

3. Построение и Обучение Сверточной Нейронной Сети (CNN)

- **Functional API:** Использовали более гибкий функциональный API для построения CNN, что позволяет создавать нелинейные архитектуры.
  - `Conv2D`: Добавили сверточные слои для извлечения признаков из изображений.
  - `AveragePooling2D`: Применили слои пулинга для уменьшения размерности карт признаков.
  - `Flatten` и `Dense`: Соединили сверточные слои с полносвязными для классификации.
- **Компиляция и обучение CNN:** Скомпилировали модель с оптимизатором `'adam'` и обучили её аналогично FCNN.
- **Сравнение производительности:** Визуализировали истории обучения обеих моделей, показав значительное улучшение точности CNN по сравнению с FCNN.
- **Анализ CNN:**
  - Предсказания и матрица ошибок показали гораздо лучшую производительность CNN.
  - **Визуализация активаций:** Создали модель для визуализации карт признаков после каждого сверточного слоя, что позволило увидеть, как сеть обрабатывает изображение и извлекает высокоуровневые признаки.

4. Дальнейшие Шаги и Улучшения